

Become a

Global AI Engineer

What it means, why it matters, and how to get there.

A masterclass for software engineers at every stage



few hours

of deep content

7 layers

of the AI-native stack

1 goal

your career edge

Scaler Academy • 2026

What We're Covering Today

PART 1

The Earthquake

Why the market has already moved

PART 2

Who Is the AI-Native Engineer?

The 7-layer stack

PART 3

Agent Orchestration

The centrepiece — how to direct AI teams

PART 4

The New Interview

What top companies are testing right now

PART 5

Advanced Skills

RAG, agents, LLMOps, evals

PART 6

Taste & Last Mile

What separates functional from incredible

PART 7

Your Roadmap

Where to go from here

1000 applications sent. 2 responses.

A Berkeley CS graduate applied to a thousand companies. Not rejected — just never heard back. This is the reality for engineers who haven't adapted.

IN THE NEXT FEW HOURS

Understand what AI-native
actually means

See the new interview format
top companies run

Learn agent orchestration,
RAG & LLMOps

Get a roadmap you can
act on tomorrow

The Perfect Storm

Three forces collided simultaneously — and changed tech hiring permanently.

RIVER 1

COVID Overhiring

2020–22: companies bloated headcount by 30–40%, then cut 20–30% in 2023. Thousands of roles disappeared overnight.

RIVER 2

CS Graduate Surge

CS enrolment has doubled–3x in the last 15 years. More junior engineers entering the market than ever before. Supply exploded.

RIVER 3

AI Goes Mainstream

Employers now ask: 'Do I hire 5 juniors, or 2 AI-native engineers who cover the same output?' The calculus shifted overnight.

THE CONFLUENCE

'Hire fewer people — but only if they're AI-native.' The hiring bar has permanently shifted.

The Market is Splitting — Not Shrinking

Engineers with 2+ AI skills earn 43% more than peers at the same YOE. The market is not shrinking — it is splitting.

SIDE A — Old-Mode Engineer

Writes boilerplate, follows ticket queue

Performance measured in lines & tickets

Resume: React, Java, Spring Boot

Compressed by AI + offshore simultaneously

Flat salary, shrinking demand

SIDE B — AI-Native Engineer

Ships outcomes; 3–5 agents in parallel

Performance = features delivered, impact

Resume: RAG, agents, evals, LLMOps

Premium demand across all experience levels

40–90 LPA in India • \$300–500k+ in the US

The Pay Premium Is Real

Same YOE. Same college. Different skill set. Dramatically different outcome.

Role / Level	Traditional SWE	AI-Native SWE	Premium
Junior 0-2 YOE (India)	₹4-12 LPA	₹6-18 LPA	+30-50%
Mid 3-6 YOE (India)	₹14-28 LPA	₹25-50 LPA	+40-80%
Senior 7-12 YOE (India)	₹30-50 LPA	₹50-90 LPA	+60-80%
Principal / Staff (India)	₹70-90 LPA	₹1-1.5 Cr	+50-70%
Senior Engineer (US)	\$160-220k	\$300-500k+	+50-120%

Sources: Glassdoor Bangalore 2026, SalaryExpert India, Metaintro 2026 data.

“Will AI Take My Job?”

Let's answer this directly.

The Excel Moment — 1985

When Excel shipped, accountants panicked. “The machine will do my maths. I’m done.” What actually happened? Bookkeepers who refused Excel got priced out. Accountants who embraced it became CFOs — they could now model 100 scenarios before lunch. Excel didn’t replace accountants. It replaced the bottom tier and accelerated the top tier.

The same dynamic is happening right now in software engineering.

10×

AI-native senior — orchestrates 3–5 agents

5×

AI-assisted mid — directs + reviews AI output

2×

AI-curious — uses suggestions, mostly manual

1×

Traditional SWE — writes every line manually

What “AI-Native Engineer” Actually Means

“At its core, the AI-native engineer has a strong foundation in traditional programming, system design, and algorithmic thinking — but is also very competent at using agentic workflows.”

— AI expert

WING 1 — CS Foundations

- DSA • Algorithms
- System design
- Databases, caching, queues
- Debugging unfamiliar code
- Test-writing discipline

WING 2 — AI Craft

- Prompting fluency
- Agent orchestration
- RAG / vector stores
- Evals & guardrails
- LLMOps & tracing

Ships outcomes, not tickets — 1 engineer ≈ output of 3–5 traditional SWEs

A Realistic Friday — What Does One Actually Do?

10:00

Sorts 3 tickets: AI-shaped vs Human-shaped. Two agents launched. No code written yet — she is the orchestrator.

11:40

Agent hallucinates an ORM method. She spots it in the log, pastes the real signature into a 2-line correction. Agent recovers in 90 seconds.

13:00

PR review of agent output. 120 lines. Accepts 90%. Rejects one section — writes the missing test. That becomes the new contract.

15:00

Architectural ticket: RAG-based help search. Agent can't own this. She sketches design, writes mini-RFC, drops in Slack.

16:30

Context-switch: Ticket A merged, Ticket B clarified, Ticket D handed to Agent 3. Managing 3 agents like an engineering manager.

19:00

4 merged PRs. 1 RFC. 1 eval baseline. Her peer next door (traditional SWE): 1 PR, 2 comments.

The 7-Layer Stack — Your Career Map

Every layer you add multiplies your value. Every layer you skip is a blindspot.

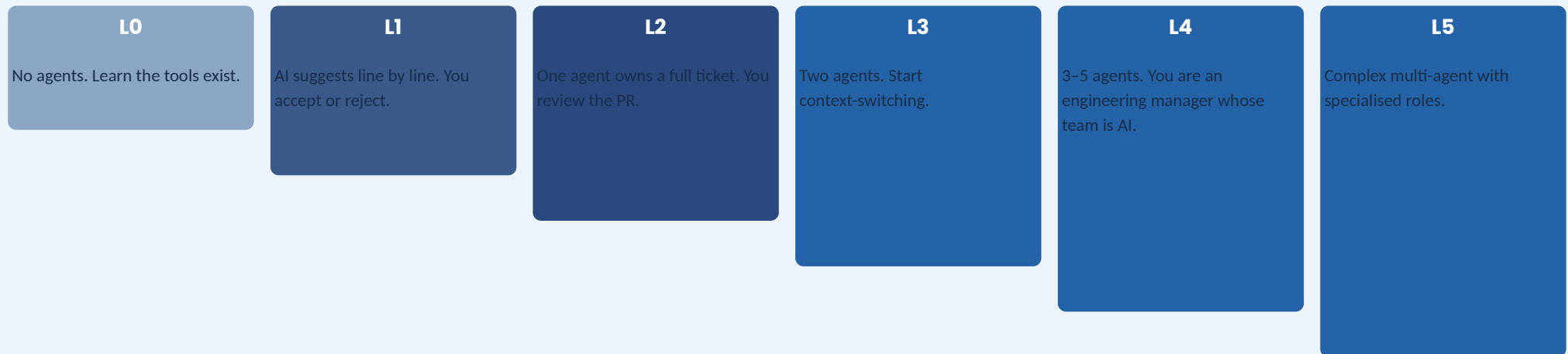
L7	TASTE	Functional vs Incredible. The last-mile instinct that gets you promoted.
L6	LLMOps	Tracing, eval suites, cost dashboards, guardrails, model routing.
L5	Multi-Agent	Tool use, ReAct loop, parallel agents, context switching, error recovery.
L4	RAG Systems	Embeddings, chunking, vector stores, re-ranking, context window design.
L3	LLM APIs	Tokens, streaming, function calling, structured output, cost control.
L2	Prompting	System vs user roles, few-shot, chain-of-thought, output constraints.
L1	CS Foundations	DSA, system design, debugging, reading unfamiliar code. The permanent floor.

The #1 Mistake: Starting With 10 Agents

"Someone will hear that an Anthropic engineer does 10 agents at once and think they should start doing 10 agents at once — that's the wrong outcome to emphasize. Build it up piecemeal."

— AI expert

The Piecemeal Ladder

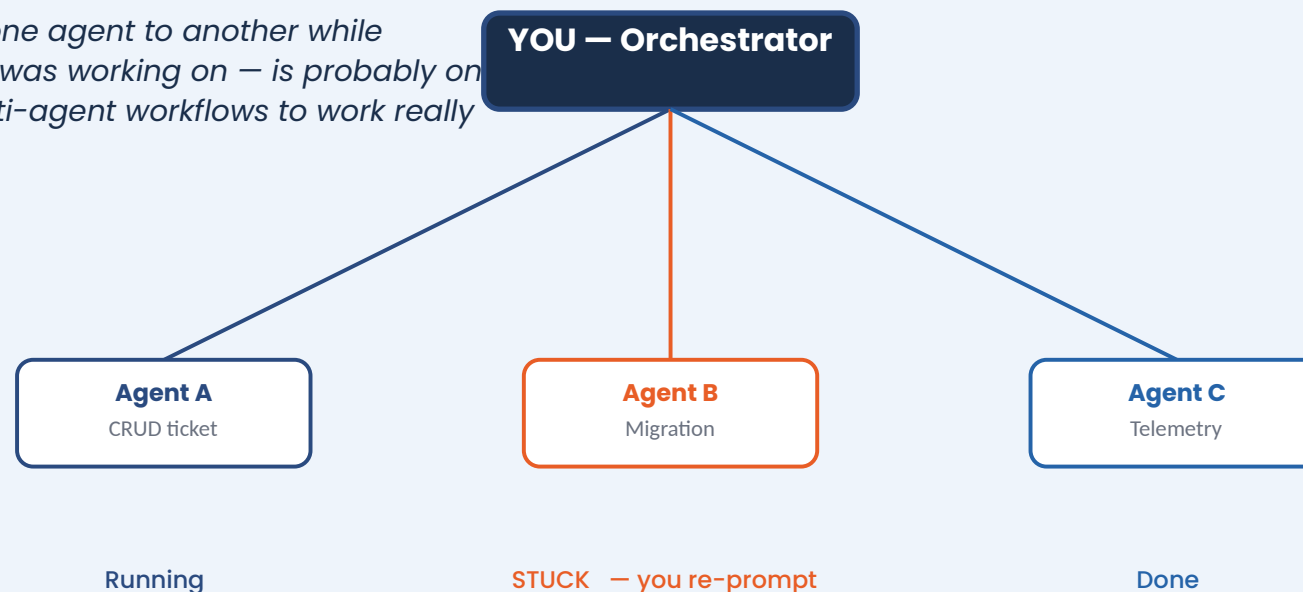


Rule: do not skip a level. Compounding errors punish skippers. See next slide.

The Hidden Core Skill: Context Switching

"That switching — going from one agent to another while understanding what each one was working on — is probably one of the core skills of getting multi-agent workflows to work really well."

— AI expert



For seniors in the room:

If you've led a team of 3–5 engineers, you already have this muscle. Context switching across agents is isomorphic to context switching across junior engineers. Your experience transfers directly. You are closer to Level-5 than any fresher in this room.

The Agent-Friendly Codebase

Agents are brilliant amnesiac interns. The codebase must teach them every morning.

1

Tests = Contracts

Agents can only operate on explicitly defined contracts. A test is a contract: “this function must return X given Y.” No tests = no contracts = agent makes random assumptions. Aim for 80%+ coverage before introducing any agent.

2

README Code Consistency

READMEs go stale within a sprint. Agent reads README, follows it, code says the opposite — picks arbitrarily, installs the wrong pattern. Fix: README accuracy is a PR-merge requirement, not a nice-to-have.

3

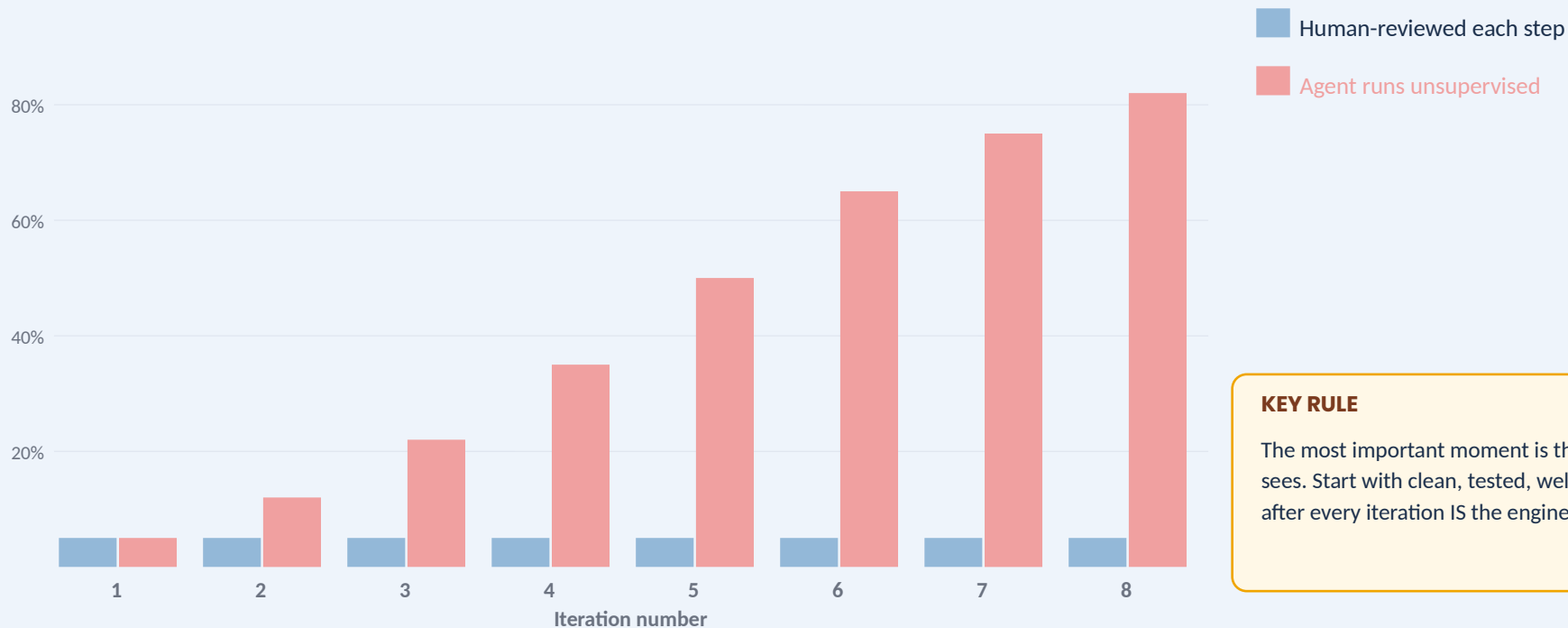
One Way to Do a Thing

Three ways to instantiate your main object? Agent picks the wrong one. Three DB query patterns? Same problem. Consistency beats cleverness in the agent era. Pick one idiom. Enforce via linter.

Why Agents Make Spaghetti

"If an agent has one misunderstanding in the code and then sees that misunderstanding it created in step one, it can double down and create another error in step two — it magnifies it."

— AI expert



KEY RULE

The most important moment is the FIRST thing the agent sees. Start with clean, tested, well-named code. The review after every iteration IS the engineering job now.

Anthropic Rewrites Their Own AI... Using That Same AI. Every Week.

"Even a team like Anthropic, building such an amazing piece of software, basically rewrites Claude Code every week or two using Claude. They are constantly rewriting their own software with software they've built. They themselves are figuring things out as they go."

— AI expert

Nobody has it figured out.

Not even the companies building the best AI tools. They're experimenting too.

Iteration is the method.

The mode of work has shifted from 'understand then build' to 'experiment, observe, iterate.'

Start now, not later.

Waiting until you fully understand AI before using it is the surest way to fall behind.

The Interview Format Has Changed Fundamentally

Top companies have moved beyond the classic LeetCode format. Here's what they're testing now.

Old Interview Format

Single function, blank file

30–50 lines of code total

Algorithm memorisation

No tools allowed

Evaluated: correctness + speed

Silent coding is fine

New AI-Enabled Format

Multi-file codebase with 100s of existing lines

Several hundred lines across multiple files

Reading unfamiliar code at speed

AI assistant provided — use it or don't

Evaluated: judgement, AI use, verification

You must narrate your reasoning throughout

The Three-Phase Interview Arc

Knowing this lets you pace yourself. Most candidates burn out in Phase 1.

Phase 1

Bug Fix

You are dropped into an existing multi-file codebase. A bug exists somewhere. Find it and fix it using AI assistance + your own knowledge and code-reading skills.

~15 min

Phase 2

Feature Implementation

Add a new feature on top of the now-working codebase. AI helps generate boilerplate; you direct every decision and verify every output. Most candidates reveal their ceiling here.

~15 min

Phase 3

Optimisation & Scale

Larger inputs, edge cases, production-readiness. Concurrency questions. This is where senior versus staff-level candidates separate. The AI cannot carry you here.

~15 min

What Actually Separates Pass from Fail

"The candidates who passed weren't better at prompting. They knew what to build, caught what the AI got wrong, and could defend every decision they made."

Codebase Orientation	AI Use	Narration	Verification	Algorithm Choice
FAILS Reads every file top to bottom; runs out of time	FAILS Prompts blindly, accepts all output without checking	FAILS Silent — interviewer has no window into thinking	FAILS Submits AI output without running tests first	FAILS Lets AI pick the algorithm for the problem
PASSES Gets entry point, data flow, key classes in 5 min	PASSES Prompts with context, reads output critically, rejects errors	PASSES Talks through reasoning live while prompting AI	PASSES Writes tests first, runs AI output against them	PASSES Recognises the right pattern; AI just implements it

Guaranteed fail: prompt your way to success without writing any code or reviewing AI output. CS foundations still matter.

RAG: Giving Your AI a Memory

Without RAG: AI answers from training data. With RAG: AI answers from your documents, grounded and citable.

1

Embed Query

User question is converted into a dense vector — a list of numbers representing meaning, not keywords.

2

Vector Search

The vector is compared against your indexed knowledge base. Top-5 most semantically similar chunks returned.

3

Re-rank

A re-ranker re-scores the 5 candidates for TRUE relevance — not just similarity. Returns the best 2-3 chunks.

4

Augment + Call LLM

Best chunks + user question + system prompt sent to the LLM. Answer is now grounded in YOUR data.

The 3 RAG Failure Modes to Know:

Chunking failure:

Too small = context lost; too large = noise. Use 256-512 tokens with 20% overlap.

Embedding mismatch:

Query model $\neq$ index model. Silent and devastating. Always use the same model for both.

No re-ranker:

Vector similarity $\neq$ relevance. Ship a re-ranker from Day 1, not as a future optimisation.

How AI Agents Actually Think: The ReAct Loop

Think. Act. Observe. That's the loop that powers every capable AI agent.

THINK

Agent reasons about what information or action is needed next to progress toward the goal.

EXAMPLE

Example: "I need to query orders from the last 7 days"

ACT

Agent calls a tool: database query, web search, file write, or any custom tool you've defined for it.

EXAMPLE

Example: `call db_query({table: "orders", days: 7})`

OBSERVE

Tool returns output. Agent updates its understanding. Goal reached? No → new THINK. Yes → deliver final answer.

EXAMPLE

Example: 47 rows returned. 3 cancelled. Now compute refund totals.

Advanced: Agents can call multiple tools simultaneously (parallel tool calling) — ~4x throughput. Use guardrails on all destructive actions.

Multi-Agent System Design Patterns

Staff-level engineers need to know when to apply each pattern and why.

Orchestrator Specialists

Central orchestrator receives the goal, makes a plan, and routes sub-tasks to specialist agents (Research, Code, Review, Deploy). All run in parallel where possible.

USE WHEN:

Tasks can be parallelised and specialised. Best for complex feature development.

Pipeline (Sequential)

Linear chain: each agent's output becomes the next agent's input. Parse → Validate → Enrich → Write.
No step starts until the previous completes.

USE WHEN:

Strict ordering dependency. Add retry + fallback at every step to prevent cascades.

Critic-Actor Loop

Actor generates output. Critic evaluates and scores. If score < threshold, Actor revises based on critique. Loop continues until quality bar is met.

USE WHEN:

Where output quality must be very high. Always cap at 3-5 iterations to prevent infinite loops.

LLMOps: The Unglamorous Skill That Gets You Hired

DevOps was 'too boring' in 2014. By 2018, DevOps engineers were the most in-demand people in the building. LLMOps is 2024's DevOps.

Tracing

Every LLM call logged: prompt in, response out, latency, cost, model version. Searchable and visualisable.

WITHOUT IT:

Without it: you're flying completely blind in production.

Evals

100+ query-answer pairs, hand-labelled. Run every deploy. Catch regressions before users find them. RAGAS metrics: faithfulness, relevancy, context precision.

WITHOUT IT:

Without it: regressions discovered by angry users, not you.

Cost Dashboard

Token cost per user session, per feature. Which prompts are bloated? A 25x cost difference exists between models — smart routing cuts costs by 70%.

WITHOUT IT:

Without it: bill shock and unexplained 10x cost spikes.

Guardrails

Prompt injection detection. PII redaction. Output filtering. Toxicity scoring. One malicious prompt can hijack an entire agent without these.

WITHOUT IT:

Without it: one bad input can compromise your whole system.

Functional vs Incredible Software

"Taste-building happens in that last mile, where you go spend the extra work to expand the feature, make it more robust, make more things possible in the application. The students that did the best are literally now building startups around their projects."

— AI expert

Functional Software	Incredible Software
- Meets acceptance criteria - Works on the happy path	Catches edge cases the ticket never mentioned Handles the sad path with graceful, informative errors
- Ships by deadline	Ships by deadline AND writes: "here's what I'd improve next"
- Uses the obvious solution	Benchmarks two approaches, picks correctly, documents why
- Code readable to the author	Code readable to the intern joining next month
- Closes the Jira ticket	Closes the ticket AND opens a new one for the opportunity spotted while building

Why Freshers Have a Superpower Right Now

"Someone coming to the industry for the first time is like a sponge. They're not scarred by how hard healthcare is. They just see a problem and think, why don't I go try and do it? There's a good naivety to how young people think — which is perfect for a startup founder."

— AI expert

No industry scars

Senior engineers resist new tools because they have 15 years of muscle memory saying "the old way works." You don't have those scars. That's an advantage.

Fastest to level up

People learning these AI skills for the first time end up being the most nimble and fastest at using them. Early-adopter advantage is real.

The arrogance of possibility

"That confidence — almost like arrogance — of seeing any problem and thinking software is the solution. That's one of the most powerful properties of CS developers." — AI expert

"Experimentation is the name of the game. Start now. You'll figure it out as you go — just like everyone else."

Your Learning Roadmap — Layer by Layer

Every section of this masterclass maps directly to a real, learnable skill. Here's where each one lives.

Layer	Skill Area	What You Learn	What You Build
L1	CS Foundations	DSA + System Design	Optimised search, scalable architecture
L2	Prompting	Prompt Engineering	Chain-of-thought systems, output constraint design
L3	LLM APIs	GenAI Fundamentals	Multi-modal app, structured output pipeline
L4	RAG Systems	RAG & Vector Databases	Company knowledge-base chatbot
L5	Agents	Agentic AI	Multi-agent orchestration system
L6	LLMOps	LLMOps & Evals	Tracing dashboard + eval suite
L7	Taste	Capstone (industry-grade)	Full-stack AI product, shipped end-to-end

AI is built into every module from Day 1 — not bolted on. Curriculum updates as the field evolves. Always.

The question isn't whether you want to be that engineer.

You're here, so I know you do. The only question is: how soon do you start?

"Fundamentally, what software teaches you is how to think about building a complex system. AI doesn't replace that. AI amplifies it." — AI expert, Stanford

What to do next:

- Reflect on which layer of the 7-stack is your biggest gap
- Open an AI coding tool on a real work ticket this week
- Come back to this deck — the concepts compound over time